

Technical Architecture Document

TigerGraph GraphRAG Hackathon — Round 3: Generalizable Context Optimization Challenge

Team: Sri Lakshmi Polavarapu **Dataset:** sp100 SEC filings (400 filings, 100 S&P 100 companies, ~18.7M tokens) **Evaluation set:** 50 organizer-provided questions across four difficulty tiers (A: single-hop, B: two-hop, C: multi-hop/corpus-scan, D: cross-document/temporal)

1. Overview

This system implements and compares three retrieval pipelines against the same sp100 dataset and question set: an LLM-only baseline, a Traditional RAG baseline (ChromaDB), and an Optimized GraphRAG pipeline built on TigerGraph Savanna. The architecture is a direct evolution of a Round 1 submission (885 arXiv papers), adapted to a heterogeneous, organizer-provided financial dataset without changing the underlying pipeline logic, per Round 3's generalization test.

2. Language Model

- **Model:** Google Gemini 2.5 Flash
 - **Generation config:** `temperature=0` for all three pipelines, so differences in output reflect retrieval strategy rather than sampling randomness
 - **Same model and configuration used across all three pipelines** for a controlled comparison
-

3. Embedding Model

- **Model:** `all-MiniLM-L6-v2` (sentence-transformers), 384-dimensional embeddings, runs locally (no API cost)
- **Used identically by Traditional RAG and GraphRAG**, so the comparison isolates the contribution of graph structure rather than embedding quality differences
- **Known constraint:** this model has a ~256 token (~1,200 character) max sequence length and silently truncates anything beyond that. This shaped a key architecture decision described in Section 6.

4. Vector Database per Pipeline

Pipeline	Vector Store	Graph Store
LLM-Only	None (no retrieval)	None
Traditional RAG	ChromaDB (persistent, local)	None
GraphRAG	TigerGraph Savanna (native vector attribute)	TigerGraph Savanna

GraphRAG uses TigerGraph Savanna as **both** the graph database and the vector database, per Round 3's explicit requirement. `Filing` vertices carry a native vector attribute (`content_embedding`), 384-dim, COSINE metric), populated during ingestion. A GSQL query (`vectorSearchFilings`) performs native similarity search over this attribute at query time, no external vector store is used for GraphRAG.

5. TigerGraph Graph Schema

```
Company (vertex)
- ticker: STRING (primary ID)

Filing (vertex)
- filing_id: STRING (primary ID, format: <TICKER>_<filename>)
- filing_type: STRING (8-K, 10-K, DEF14A)
- filing_date: STRING
- content: STRING (excerpt used as displayed/cited evidence)
- content_embedding: VECTOR(384), COSINE metric

Topic (vertex)
- name: STRING (primary ID, a financial term or filing type)

FILED (undirected edge)
Company -- Filing

COVERS_TOPIC (undirected edge)
Filing -- Topic
```

RELATED_TO (undirected edge)

Topic -- Topic (reserved for future topic-co-occurrence linking)

This schema replaced Round 1's `(Paper / Author / Concept)` schema, which was built around arXiv-specific structure (title/abstract/authors) that doesn't exist in SEC filings. `(Company / Filing / Topic)` reflects the actual entities and relationships present in the sp100 dataset.

6. Entity and Relationship Extraction Methodology

Companies and Filings: extracted directly from file structure. The sp100 dataset is organized as `(data/sp100_dataset/<TICKER>/<TYPE>_<DATE>.txt)`, so ticker, filing type, and filing date are parsed from the path and filename, no NLP extraction needed for these fields.

Topics: extracted via keyword matching against a fixed list of ~30 financial terms (revenue, net income, dividend, litigation, acquisition, etc.), plus the filing type itself tagged as a topic. This mirrors Round 1's approach (keyword matching against an AI/ML concept list) adapted to the financial domain.

Filing embeddings — a genuine architecture problem and fix: Initial ingestion embedded `(content[:2000])` (the first 2,000 characters) of each filing. Because every SEC filing opens with a near-identical cover page (company address, filing type, legal boilerplate), embeddings computed this way clustered by filing *format* rather than filing *content* — a smoke test showed a query about Apple's net sales retrieving mostly Uber, Walmart, and Netflix filings.

Widening the embedded window did not help: `(all-MiniLM-L6-v2)`'s ~1,200 character effective limit meant a wider slice was silently truncated to the same effective content. The fix was to **search for financial section markers** (`("results of operations")`, `("net sales")`, `("item 7")`, `("management's discussion")`) within each filing and embed starting from the first match, falling back to the post-cover-page offset if no marker is found. This moved retrieval from matching filing structure to matching actual financial content, and was confirmed via smoke test and a full 50-question re-run (see Section 9 for the measured accuracy impact).

7. Chunking and Indexing Strategies

Traditional RAG (ChromaDB): documents are split into 500-token chunks with 50-token overlap (`(preprocessing/chunker.py)`), a straightforward sliding-window chunker that is dataset-agnostic — no changes were needed when swapping from arXiv papers to SEC

filings. Given the sp100 dataset's much longer average document length (~46,695 tokens/filing vs. round 1's short abstracts), this produced ~41,695 chunks across 400 filings, roughly 40x round 1's chunk volume.

GraphRAG (TigerGraph): does not chunk documents. Each `(Filing)` vertex stores one excerpt (up to 3,000 characters, sourced from the financial section identified in Section 6) and one embedding vector. Retrieval works at the filing level, not the chunk level — a deliberate difference from Traditional RAG's dense chunking, since GraphRAG's structured company/topic/filing relationships are meant to substitute for fine-grained chunk-level recall.

8. Retrieval, Reranking, Filtering, and Context-Optimization Techniques

Traditional RAG: top-k (k=5) cosine similarity search over ChromaDB chunk embeddings. No reranking or filtering beyond the initial similarity ranking.

GraphRAG — hybrid retrieval:

1. **Vector search:** the question is embedded with the same model used for filings, and TigerGraph's native `(vectorSearch())` GSQL function returns the top-k (k=5) most similar `(Filing)` vertices by cosine distance.
2. **Graph traversal:** financial topics and ticker symbols are extracted from the question (keyword matching), and the graph is traversed `(Topic -COVERS_TOPIC- Filing)` and `(Company -FILED- Filing)` to find additional relevant filings via explicit relationships rather than semantic similarity alone.
3. **Merge:** both result sets are combined, deduplicated by `(filing_id)`, and capped at 8 total results before being passed to the LLM.

This hybrid approach — vector search for semantic recall, graph traversal for explicit relationship-based recall — is what satisfies Round 3's requirement that GraphRAG make "meaningful use of graph structure" beyond a vector-only pipeline: storing disconnected chunks with no graph traversal would not have met this bar, and a pure keyword/topic-matching version (tested during development, see Section 9) measurably under-performed the hybrid version on accuracy.

No LLM-based reranking, filtering, or compression step is used by either retrieval pipeline in the current implementation — this is disclosed as a limitation, not a hidden difference.

9. Evaluation Tools and Frameworks

- **BERTScore** (`(bert-score==0.3.13)`), pinned config: `(model_type="roberta-large")`,

`(lang="en")`, `(rescale_with_baseline=True)`, `(idf=False)`, computed against the official reference answer (never retrieved context), per Round 3's exact comparability requirement.

- **Strict pass/fail judge:** rejects hedged or non-answer responses (phrases like "has not yet reported," "cannot determine") outright, and otherwise requires $\geq 75\%$ keyword coverage of the reference answer.
- **Graded quality score:** four tiers — `fully_correct` ($\geq 85\%$ coverage), `mostly_correct` ($\geq 60\%$), `partially_correct` ($\geq 30\%$), `incorrect` — with the same hedge-detection rule forcing "incorrect."
- **Lenient judge:** kept from Round 1 methodology ($\geq 40\%$ keyword coverage) for direct comparison across rounds.
- **Citation and evidence-quality scoring:** citation rate (completeness — fraction of retrieved evidence actually cited), citation support rate (correctness — whether cited evidence textually overlaps the answer), and duplicate evidence rate (fraction of retrieved evidence that is near-duplicate within the same question's result set).
- **Tier breakdown:** all of the above, broken out by question difficulty tier (A/B/C/D), to assess consistency across question and document types.

Measured impact of the vector search + embedding fix (full 50-question re-runs, GraphRAG):

Stage	Lenient Pass	Strict Pass	Fully Correct	BERTScore F1
Keyword/topic matching only (no vector search)	18%	2%	2.0%	-0.1908
+ Native vector search added, embedding on <code>content[:2000]</code>	24%	10% ¹	4.0% ¹	-0.1800 ¹
+ Embedding fixed to target financial section markers	34%	10%	4.0%	-0.1312

¹ Intermediate values from the boilerplate-embedding version before the marker-based fix.

10. Agentic Frameworks, Orchestration Tools, and Libraries

- `google-genai` — Gemini API client
- `pyTigerGraph` — TigerGraph Savanna connection, GSQL execution, REST++ vertex/edge upserts and queries

- `chromadb` — Traditional RAG vector store
 - `sentence-transformers` — local embedding model
 - `tiktoken` — token counting (cl100k_base encoding) for context/system/ question token accounting
 - `bert-score` — BERTScore F1 computation
 - `huggingface_hub` (`InferenceClient`) — LLM-as-judge inference (`microsoft/Phi-3-mini-4k-instruct`), with local keyword-based fallback if the API is unavailable
 - `streamlit` + `plotly` — comparison dashboard
 - No agentic orchestration framework (LangChain, LlamaIndex, etc.) is used — each pipeline is a direct, explicit sequence of retrieval → prompt construction → generation, no agent loop or tool-calling framework is involved.
-

11. Known Limitations and Disclosed Gaps

In the interest of the transparency Round 3 explicitly asks for:

- No LLM-based reranking, filtering, or compression step exists in either retrieval pipeline yet.
- No dev/held-out test split has been implemented; all 50 questions were used directly for both any informal tuning and final evaluation.
- The full 3x-repeated held-out evaluation (for mean/variance reporting) has not yet been executed, though the runner (`evaluation/reproducibility.py`) is implemented and ready to run.
- Ablation analysis (`evaluation/ablation.py`, vector-only vs. graph-only modes) is implemented but has not yet been executed against the current (fixed-embedding) version of GraphRAG.
- GraphRAG's accuracy, while substantially improved, still trails Traditional RAG's on this dataset. The efficiency gains are real and measured, but per Round 3's own validity rule, they should be read alongside this remaining accuracy gap rather than in isolation.